

SISTEMAS OPERATIVOS

08011442

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



Servicios POSIX
Gestión de procesos

Contenido

1. Identificación de un proceso
2. Entorno de un proceso
3. Creación de procesos: fork()
4. El valor de retorno de fork()
5. Terminación y espera: exit, wait, waitpid
6. Procesos zombis y huérfanos
7. Cambio de programa: la familia exec
8. Ejercicios



Identificación
y entorno del proceso

Identificación de un proceso

- Cada proceso tiene un **identificador** único, el **pid**, de tipo `pid_t` (un entero con signo).
- POSIX ofrece servicios para consultar el pid propio y el del proceso padre.

pid_t getpid(void);

Include	`<unistd.h>`
Retorna	El pid (entero positivo) del proceso que hace la llamada. Nunca falla.

pid_t getppid(void);

Include	`<unistd.h>`
Retorna	El pid del proceso padre: el que creó a este proceso. Nunca falla.

Ejemplo: identificación de un proceso

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    printf("pid:%d ppid:%d\n", getpid(), getppid());
    return EXIT_SUCCESS;
}
```

```
$ gcc -Wall -o getpid getpid.c
$ ./getpid
pid:1350784 ppid:1350782
```

- Cada ejecución muestra un **pid** distinto: lo asigna el kernel al crear el proceso.
- El **ppid** es el pid de quien lo lanzó (aquí, la shell).

Ejecute el programa varias veces: el **pid** cambia en cada ejecución, mientras que el **ppid** (la shell) se mantiene.

Entorno de un proceso

- El **entorno** de un proceso es la lista de variables que se le pasan al comenzar su ejecución.
- Es accesible mediante la variable externa `environ`: `extern char **environ;`
- Es un vector de punteros a cadenas de la forma `nombre_variable=valor`, terminado en `NULL`.
- POSIX define, entre otras, estas variables estándar:

Variable	Significado
`HOME`	Directorio personal (home) del usuario asociado al proceso
`LOGNAME`	Nombre de usuario asociado al proceso
`PATH`	Rutas donde se buscan los ejecutables
`TERM`	Tipo de terminal
`TZ`	Zona horaria

Consultar una variable: *getenv*

```
char *getenv(const char *name);
```

Busca `name` en el entorno del proceso y devuelve un puntero a su valor.

Include	`<stdlib.h>`
Retorna	**Éxito:** puntero a la cadena de valor (no debe modificarse ni liberarse). **Falla:** `NULL` si la variable no existe.
Argumentos	• ***const char *name***: nombre de la variable de entorno a consultar.

Consultar una variable inexistente (por ejemplo `k2r`) no es un error: `getenv` simplemente devuelve `NULL`.

Ejemplo: recorrer y consultar el entorno

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char **environ;

int main(void) {
    /* recorrer toda la lista de variables de entorno */
    char **list = environ;
    char *var;
    while (*list != NULL) {
        printf("%s\n", *list);
        list++;
    }
    /* obtener el valor de una variable específica */
    var = getenv("LOGNAME");
    printf("%s\n", var != NULL ? var : "(no definida)");
    return EXIT_SUCCESS;
}
```

- `environ` termina en `NULL`: se recorre hasta `*list == NULL`.
- `getenv` devuelve `NULL` si la variable no existe: hay que comprobarlo antes de usarlo.

```
$ ./entorno
SHELL=/bin/bash
PWD=/home/usuario/practicas
LOGNAME=usuario
HOME=/home/usuario
PATH=/usr/local/bin:/usr/bin:...
...
usuario
```

Salida recortada: la lista completa es larga. La última línea es el valor de LOGNAME.



Creación, terminación y espera de procesos

Creación de un proceso: *fork*

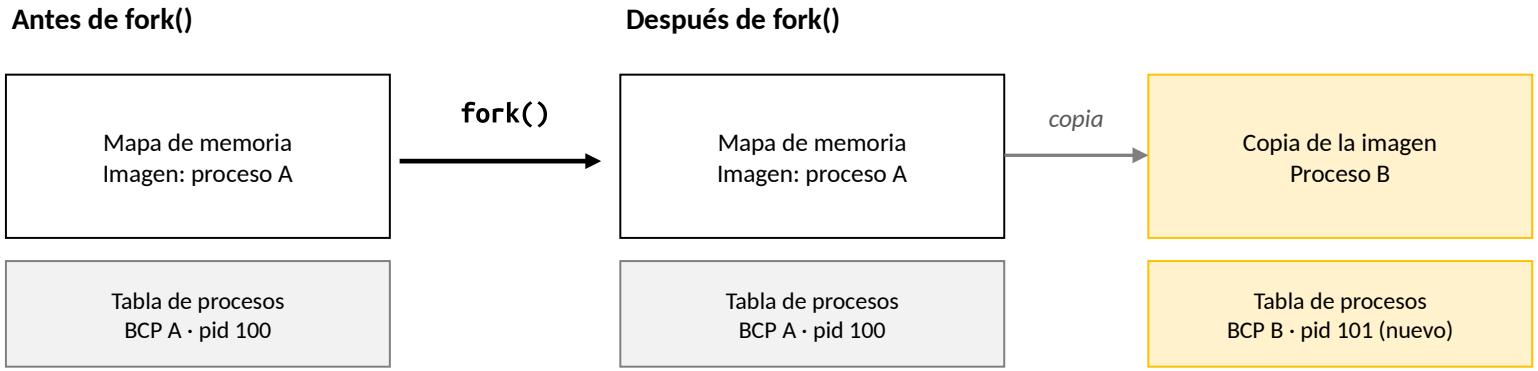
```
pid_t fork(void);
```

Crea un proceso nuevo (el **hijo**) clonando la imagen de memoria del proceso que llama (el **padre**).

Include	`<unistd.h>`
Retorna	**Al padre:** el `pid_t` del hijo recién creado. **Al hijo:** `0`. **A cualquiera, si falla:** `-1` (no se crea ningún hijo).

- Tras `fork()`, padre e hijo son procesos **independientes** que ejecutan el mismo código a partir de la instrucción siguiente a la llamada.

Copia de la imagen de memoria con *fork*



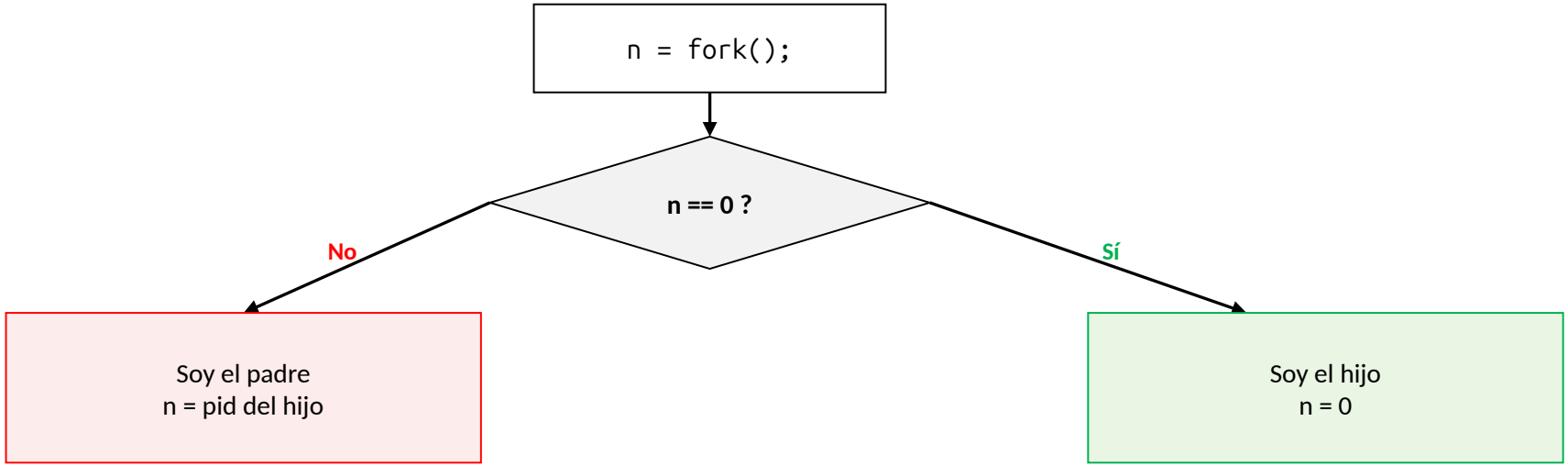
El hijo (B) recibe una copia de la imagen de memoria del padre y una entrada nueva en la tabla de procesos, con un pid propio; el padre conserva la suya sin cambios.

- El texto (código), los datos y la pila de ambos procesos son iguales en el instante de la creación.
- Ambos tienen el mismo valor de registros en su BCP: el contador de programa (CP) apunta a la instrucción siguiente a `fork()` en los dos.

Diferencias entre el proceso padre y el hijo

- Ambos tienen el mismo código, datos y pila en el momento de la creación (la memoria del hijo es una copia); por eso los dos continúan justo después de la llamada a `fork()`.
- Cada uno tiene su propio identificador (`pid`); el hijo conoce el pid de su padre.
- Aunque el contenido inicial es igual, cada proceso tiene sus propios segmentos de memoria: escribir en uno no afecta al otro.
- El tiempo de ejecución del hijo se reinicia a 0.
- El hijo no hereda alarmas ni señales pendientes del padre.
- El valor que devuelve `fork()` es distinto en cada uno de los dos procesos (a continuación).

El valor de retorno de *fork*



- Si `fork()` falla, retorna **-1** al proceso que llama y no se crea ningún hijo.
- Padre e hijo comparten los **descriptores de archivo** que el padre tenía abiertos antes de la llamada: el hijo puede leer y escribir en los mismos archivos.

Ejemplo: crear un proceso con *fork*

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* el padre ve su pid y el del hijo; el hijo, solo el suyo */
int main(void) {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return EXIT_FAILURE;
    } else if (pid == 0) {
        printf("Hijo: mi pid es %d\n", getpid());
    } else {
        printf("Padre: mi pid es %d, el de mi hijo es %d\n",
               getpid(), pid);
    }
    return EXIT_SUCCESS;
}
```

```
$ ./fork1
Padre: mi pid es 1381866, el de mi hijo es 1381867
Hijo: mi pid es 1381867
```

- El orden de los mensajes de padre e hijo no está garantizado: depende del planificador.
- Este programa resuelve el ejercicio 3 del final del tema: el padre muestra su pid y el de su hijo; el hijo, solo el suyo.

Ejemplo: crear un hijo y esperarlo con *wait*

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t pid;
    int status;
    pid = fork();              // crear un proceso hijo
    if (pid == -1) {
        perror("fork");
        return 1;
    } else if (pid == 0) {     // proceso hijo
        printf("Proceso hijo: PID = %d\n", getpid());
        sleep(2);              // esperar unos segundos
        printf("Proceso hijo terminado\n");
        exit(0);
    } else {                   // proceso padre
        printf("Proceso padre: PID = %d\n", getpid());
        printf("Esperando a que el hijo termine...\n");
        wait(&status);         // esperar al hijo
        printf("Hijo terminado con estado %d\n",
               WEXITSTATUS(status));
        printf("Proceso padre terminado\n");
        return 0;
    }
}
```

```
$ ./fork_wait
Proceso padre: PID = 1350787
Esperando a que el hijo termine...
Proceso hijo: PID = 1350788
Proceso hijo terminado
Hijo terminado con estado 0
Proceso padre terminado
```

El padre queda **bloqueado** en `wait()` hasta que el hijo termina (2 s después); `WEXITSTATUS` extrae el código de salida del entero `status`.

Ejemplo: varios hijos con ***waitpid***

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t pid[3];
    int status;
    for (int i = 0; i < 3; i++) {
        pid[i] = fork();            // crear un hijo
        if (pid[i] == -1) { perror("fork"); return 1; }
        if (pid[i] == 0) {          // proceso hijo
            printf("Proceso hijo %d: PID = %d\n",
                   i + 1, getpid());
            sleep(i + 1);           // dormir 1..3 s
            printf("Proceso hijo %d terminado\n", i + 1);
            exit(0);
        }
    }
    printf("Proceso padre: PID = %d\n", getpid());
    printf("Esperando a que los hijos terminen...\n");
    for (int i = 0; i < 3; i++) {
        waitpid(pid[i], &status, 0); // esperar al hijo i
        printf("Hijo %d terminado con estado %d\n",
               i + 1, WEXITSTATUS(status));
    }
    printf("Proceso padre terminado\n");
    return 0;
}
```

```
$ ./fork_loop
Proceso padre: PID = 1371614
Esperando a que los hijos terminen...
Proceso hijo 1: PID = 1371615
Proceso hijo 2: PID = 1371616
Proceso hijo 3: PID = 1371617
Proceso hijo 1 terminado
Hijo 1 terminado con estado 0
Proceso hijo 2 terminado
Hijo 2 terminado con estado 0
Proceso hijo 3 terminado
Hijo 3 terminado con estado 0
Proceso padre terminado
```

`waitpid(pid[i], ...)` espera a un hijo concreto, en el orden del arreglo; `wait()` habría esperado al primero que terminara, sin importar cuál.

Procesos zombis y huérfanos

- Cuando un proceso termina, se liberan su memoria y sus recursos, pero su entrada en la **tabla de procesos** no desaparece todavía: así el padre puede recoger su estado de salida.
- Mientras el padre no recoja ese estado (con `wait` o `waitpid`), el hijo terminado se llama **zombi**. Se detectan con `ps -el` (marcados con Z).
- Un **huérfano** es distinto: es un proceso cuyo padre terminó antes que él; lo adopta `init` (pid 1), que sí lo recoge cuando termina.
- Acumular zombis no agota memoria de golpe, pero es señal de un programa que no espera a sus hijos.

Ejemplo: procesos zombis

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Los hijos terminan de inmediato, pero el padre
   no llama a wait(): mientras el padre siga vivo,
   quedan como zombis en la tabla de procesos. */
int main(void) {
    int n = 3;

    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Hijo %d (pid=%d) termina\n", i, getpid());
            return 0;
        }
    }

    printf("Padre (pid=%d): %d hijos, sin wait()\n",
           getpid(), n);
    printf("Pulse Enter para terminar\n");
    getchar();   /* tiempo para observarlos con ps */
    printf("Padre termina; init adopta y recoge a los zombis\n");
    return 0;
}
```

```
$ ./zombie
Hijo 0 (pid=714257) termina
Hijo 1 (pid=714258) termina
Padre (pid=714256): 3 hijos, sin wait()
Pulse Enter para terminar
Hijo 2 (pid=714259) termina
```

En otra terminal, antes de pulsar Enter:

```
$ ps -o pid,ppid,stat,comm -C zombie
    PID    PPID STAT COMMAND
 714256  714251 S+   zombie
 714257  714256 Z+   zombie
 714258  714256 Z+   zombie
 714259  714256 Z+   zombie
```

Los tres hijos aparecen en estado Z (zombi): terminaron, pero su padre (714256) aún no recoge su estado. Al terminar el padre, `init` los adopta y los recoge.

Terminación de un proceso: *exit* y *wait*

void exit(int status);

Termina el proceso: vacía los búferes de E/S, cierra descriptores y libera recursos.

Include	`<stdlib.h>`
Argumentos	• ***int status***: código de salida; el padre recibe sus 8 bits bajos (0-255) con `WEXITSTATUS`.

*pid_t wait(int *status);*

Bloquea al proceso que llama hasta que termina cualquiera de sus hijos.

Include	`<sys/wait.h>`
Retorna	**Éxito:** pid del hijo que terminó. **Falla:** -1 (`ECHILD`: sin hijos).
Argumentos	• ***int *status***: si no es `NULL`, recibe el motivo de la terminación (`WIFEXITED`, `WEXITSTATUS`…).

Esperar a un hijo concreto: *waitpid*

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Igual que `wait`, pero permite elegir a qué hijo esperar y no bloquear si aún no ha terminado.

Include	`<sys/wait.h>`
Retorna	**Éxito:** pid del hijo recogido (`0` con `WNOHANG` si ninguno ha terminado). **Falla:** `-1` (`ECHILD`).
Opciones	`0` (bloquear) o `WNOHANG` (retornar de inmediato si ningún hijo ha terminado).

pid	Espera a...
`-1`	cualquier hijo (igual que wait())
`0`	cualquier hijo del mismo grupo de procesos que el padre
`> 0`	el hijo con ese pid exacto
`< -1`	cualquier hijo cuyo grupo sea |pid|



Cambio de programa: la familia exec

Cambio de programa: la familia *exec*

- Todas comparten el prefijo `exec` y reemplazan el **código, los datos y la pila** del proceso que llama por los del nuevo programa.
- Como no se crea un proceso nuevo, el `pid` no cambia: solo cambia lo que el proceso ejecuta.

int execve(const char *path, char *const argv[], char *const envp[]);

Include	`<unistd.h>`
Retorna	**Éxito:** no retorna; el nuevo programa empieza a ejecutarse. **Falla:** `-1` y `errno`; el proceso original sigue como estaba.
Argumentos	• ***const char *path***: ruta del ejecutable. • ***char *const argv[]***: argumentos del nuevo programa (`argv[0]` es su nombre), terminado en `NULL`. • ***char *const envp[]***: entorno del nuevo programa, terminado en `NULL`.

La familia ***exec***: variantes

Función	Argumentos	PATH	Entorno
`execl`	Lista (termina en NULL)	No: ruta completa	Hereda el del proceso (environ)
`execlp`	Lista (termina en NULL)	Sí: busca en PATH	Hereda el del proceso (environ)
`execle`	Lista (termina en NULL)	No: ruta completa	Explícito: último argumento (envp)
`execv`	Vector (argv[])	No: ruta completa	Hereda el del proceso (environ)
`execvp`	Vector (argv[])	Sí: busca en PATH	Hereda el del proceso (environ)
`execve`	Vector (argv[])	No: ruta completa	Explícito: argumento envp[]

execve es la llamada al sistema; las demás son funciones de biblioteca que arman sus argumentos y terminan llamando a execve.

Convención de nombres de la familia *exec*

- El nombre de cada función combina `exec` con sufijos que indican cómo se pasan los argumentos y el entorno del nuevo programa:

Sufijo	Significa	Aparece en
`l`	Argumentos como lista, terminada en NULL	`execl, execlp, execle`
`v`	Argumentos como vector (`char *argv[]`)	`execv, execvp, execve`
`p`	Busca el ejecutable en los directorios de PATH	`execlp, execvp`
`e`	Recibe un entorno explícito (`envp[]`)	`execle, execve`

Sin **p**: hay que dar la ruta completa del ejecutable. Sin **e**: el nuevo programa hereda el entorno actual del proceso (`environ`).

Ejemplo: reemplazar el proceso con *execvp*

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t pid;
    int status;
    pid = fork();               // crear un hijo
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {             // proceso hijo
        char *args[] = {"ls", "-al", NULL};
        execvp(args[0], args);
        printf("Error al ejecutar\n"); // solo si falla
        return 1;
    }
    wait(&status);              // esperar al hijo
    printf("Proceso hijo terminado con estado %d\n",
           WEXITSTATUS(status));
    return 0;
}
```

```
$ ./exec_execvp
total 188
drwxrwxr-x  2 usuario usuario  4096 ... .
drwxrwxr-x 12 usuario usuario  4096 ... ..
-rwxrwxr-x  1 usuario usuario 16232 ... exec_execvp
-rw-rw-r--  1 usuario usuario   564 ... exec_execvp.c
...
Proceso hijo terminado con estado 0
```

Como no se da una ruta, `execvp` busca `"ls"` en PATH. Si tiene éxito, `execvp` no retorna: el *"Error al ejecutar"* solo se imprimiría si fallara.

El hijo **se convierte** en `ls` (mismo pid); el padre sigue siendo el mismo programa y lo espera con `wait`.

Ejemplo: encadenar procesos con *execl*

exec_primero.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t hijo[2];
    int i, status;
    for (i = 0; i < 2; i++) {
        hijo[i] = fork();
        if (!hijo[i]) {
            if (i == 1) {
                execl("./exec_segundo",
                      "exec_segundo",
                      "1", "2", NULL);
            }
            printf("Hijo %d\n", i);
            break;
        }
    }
    if (i == 2) for (i = 0; i < 2; i++) {
        wait(&status);
        printf("Hijo[%d] retornó %d\n",
               i, WEXITSTATUS(status));
    }
    return 0;
}
```

exec_segundo.c

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    int i;
    printf("Argumentos: %d\t", argc);
    for (i = 0; i < argc; i++) {
        printf("%s\t", argv[i]);
    }
    printf("\n");
    return -1; // a propósito: ver WEXITSTATUS
}
```

El hijo 1 se reemplaza por exec_segundo, que devuelve -1: WEXITSTATUS lo muestra como 255.

```
$ gcc -Wall -o exec_segundo exec_segundo.c
$ gcc -Wall -o exec_primero exec_primero.c
$ ./exec_primero
Hijo 0
Hijo[0] retornó 0
Argumentos: 3   exec_segundo    1       2
Hijo[1] retornó 255
```

El índice de Hijo[i] es el orden en que wait() los recoge, no necesariamente el hijo i.

Ejercicios

- **Ejercicio 1.** Escribir un programa que muestre todas las variables de entorno accesibles mediante `environ`.
- **Ejercicio 2.** Escribir un programa que, usando `getenv`, obtenga el valor de las variables de entorno `DISPLAY`, `HOME` y `PWD`. Observar qué ocurre al consultar una que no exista (por ejemplo, `k2r`).
- **Ejercicio 3.** Escribir un programa que cree un proceso con `fork`. Tras la llamada, el padre mostrará su pid y el de su hijo; el hijo mostrará solo el suyo.

Resumen

- Cada proceso tiene un `pid` único; `getpid()` y `getppid()` consultan el propio y el del padre.
- El entorno (`environ`) es una lista de variables `nombre=valor`; `getenv()` consulta una en concreto.
- `fork()` clona al proceso que lo llama: el hijo recibe `0` y el padre el pid del hijo (`-1` si falla).
- `exit()` termina un proceso; `wait()`/`waitpid()` recogen su estado. Si el padre no lo hace, el hijo queda **zombi**.
- La familia `exec` (`execl, execv, execlp, execvp, execle, execve`) reemplaza la imagen del proceso sin crear uno nuevo: mismo pid, código distinto.

Referencias y lecturas

- Kerrisk, M. *The Linux Programming Interface*, caps. 24, 25, 26 y 27 (creación, terminación, monitorización y exec de procesos). No Starch Press, 2010.
- Stevens, W. R. y Rago, S. A. *Advanced Programming in the UNIX Environment*, caps. 8 y 9. Addison-Wesley.
- Silberschatz, A., Galvin, P. y Gagne, G. *Operating System Concepts*, cap. 3: Processes.
- Páginas de manual: `man 2 fork`, `man 2 execve`, `man 3 exec`, `man 2 wait`, `man 2 getpid`.